

Machine Intellect Society of GIT

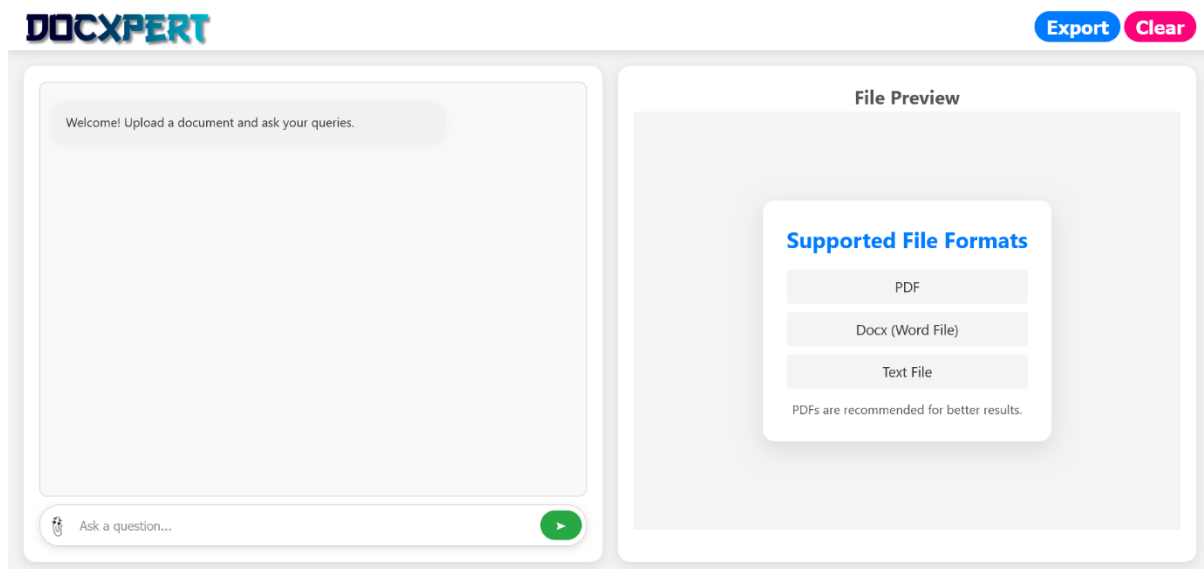
Tech-Team Task 1

Rahul Samedavar

Docxpert

Overview:

Docxpert is an AI-powered document query chatbot that **leverages Retrieval-Augmented Generation (RAG)** to provide precise and context-aware responses from PDF, Word, and Text documents. It uses OpenAI's API for language processing, ChromaDB for efficient vector storage and retrieval, and a Flask server to deliver a clean and interactive UI.



Features:

✓ Multi-File Support:

- Supports **PDF**, **Word (.docx)**, and **Text (.txt)** file formats for querying.

✓ Context-Aware Queries:

- Each new query continues from the previous context, maintaining the conversation flow.

✓ File Preview with Navigation:

- Document preview with **full view tools** for easier access.
- Auto-scroll to RAG references: Clicking on a source in the response navigates directly to the corresponding page.

✓ Temporary Sessions:

- Sessions persist as long as the server is running, allowing users to **reload the page** and continue where they left off.

✓ Exportable Chat History:

- Save the entire conversation as a **PDF file** for reference.

✓ Multi-PDF Interaction:

- Interact with **multiple PDFs** without clearing previous chat history, making cross-document queries seamless.

✓ Chat Clearing Option:

- Users can **clear the chat** and reset the context when required.

The screenshot displays the DOCXPRT application interface. At the top left is the 'DOCXPRT' logo. At the top right are 'Export' and 'Clear' buttons. The main area is split into two panels. The left panel shows a chat interface with a message box at the top stating 'File uploaded successfully: f_1.pdf'. Below it, a chat bubble contains a list of challenges: '1. Similarity between different classes and potential ambiguity in handwritten digits recognition.' and '2. Struggle in digit positioning despite excelling in testing data initially.' with sources 'p:10 p:8 p:5'. A blue button 'Challenges faced?' is next to it. Below the chat bubble is another bubble with the text 'The first challenge faced was the similarity between different classes, making it difficult for the model to discriminate between certain numbers when written by hand.' with sources 'p:10 p:8 p:10'. A blue button 'explain first' is next to it. At the bottom of the chat panel is a text input field with 'explain first' and a green send button. The right panel is titled 'File Preview' and shows a document viewer. It displays the same 'Challenges Faced:' section from the chat. Below the text is a 'DIGIT RECOGNIZER' image showing a handwritten digit '1' and a table of predictions and probabilities. The table has two columns: 'PREDICTION' and 'PROBABILITY'. The data is as follows:

PREDICTION	PROBABILITY
7	86.6%
1	27.3%
2	5.6%
4	2.7%
9	1.4%

Below the table is a 'Solution' note: 'Included Top k Accuracy metrics, with k=3, to atleast include the correct class in best 3 predictions in such cases.' At the bottom of the preview panel is the heading '2. Digit Positioning'.

Tech Stack:

- **LLM:** OpenAI GPT
- **LLM Framework:** Langchain
- **Backend:** Flask (Python)
- **Vector Store:** ChromaDB
- **UI:** HTML, CSS, JavaScript
- **File Processing:** PyMuPDF, pdf-kit, python-docx , docx2txt, reportlab

Methodology:

RAG:

1. **Embeddings:** Uses OpenAI embeddings from Langchain.
2. **File Processing and Ingestion:**
 - a. Word Files and Text Files are converted to PDF files for uniformity and easy file preview.
 - b. Now all the files are of PDF format only, this are loaded with PyMuPDFLoader.
 - c. File is split into Chunks of length 500 with overlap 150 characters using RecursiveCharacterTextSplitter from Langchain.
 - d. These chunks are embedded and stored in Individual vectorstores with chromaDB.
 - e. This vectorstores are represented by a db_name.
3. **Context based Query Processing and Retrieval :**
 - a. User Query along with chat history (last 5 only) is fed to OpenAI model to generate a **Context Aware Query**. This will be used for retrieval.
 - b. The **search** Function locates the corresponding DB and get top 3 chunks that match with the query's embeddings with threshold of 40% match.
 - c. If Context Aware Query doesn't get a match, then user Query will be used instead.

4. Response Generation:

- a. Retrieved chunks Query is fed to OpenAI models to generate response.
- b. Corresponding Page number of Retrieved Chunks are also provided as source.

Frontend Special Features:

1. Local reference to DB for persistent chats:

- a. db name representing the vectorstore name and the file name on server is stored in client browser
- b. It will be valid as long as the server is active and resets with server.
- c. This also serves as chat session id, hence chat history will be preserved.

2. File Preview:

- a. PDF files of corresponding session are loaded from backend and preview is powered by browser.
- b. Since browsers can't display word files, they are converted to PDF for preview.
- c. Sources references are Navigable, ie. You can navigate to page by clicking on source, This is achieved by dynamically manipulating page index on src.

Challenges Faced

- 1. Preview of Docx file:** browser doesn't support viewing word files, so I was trying to get workaround for it, following solutions were found
 - a. **Cloud Hosting:** Microsoft and Google can host word file previews. But they are not suitable for automation.
 - b. **Cloud Conversion:** There are many APIs to convert Word files to PDF preserving integrity. But they are not reliable as Free trial has very low rate limits.
 - c. **Python Libraries:** Python libraries like reportlab provides implementation to convert word files to PDF , but they doesn't preserve integrity, But they are enough for this project.